

Figure 2: Optimised

```
else:
    l1=len(matrix1)
    l2=len(matrix2[0])
    l3=len(matrix2)
    new_matrix = matrix(l1, l2)
    new_matrix.zero()
    for i in xrange(l1):
        for j in xrange(l2):
            for k in xrange(l3):
                new_matrix.mat[i][j] += matrix1[i]
[k]*matrix2[k][j]]
    return new_matrix
```

As you can see, rather than calling the *len* function each time in the loop, I am calling *len* on *matrix1*, *matrix2[0]* and *matrix2*, and storing the values in variables *l1*, *l2* and

l3, before using them in place of the *len* function calls. Another optimisation I used in my code is replacing all *range* function calls with the *xrange* function, a generator, which is more efficient. You can see the results of the above optimisation in Figure 2. As you can clearly see, I have reduced the execution time by roughly 2 seconds. Not much, but it is a minor improvement. Also, notice how the total number of calls has dropped from 752013 to 251014!

The problem with cProfile is that for simple code, it gives you a reasonable idea of what is going on within your code, but for more complex problems, the output becomes harder to understand. This is where Runsnake comes in.

Runsnake

Runsnake is another profiling tool, which allows you to visualise the profiled results obtained from the cProfile module. This can be useful to get the bigger picture. Use

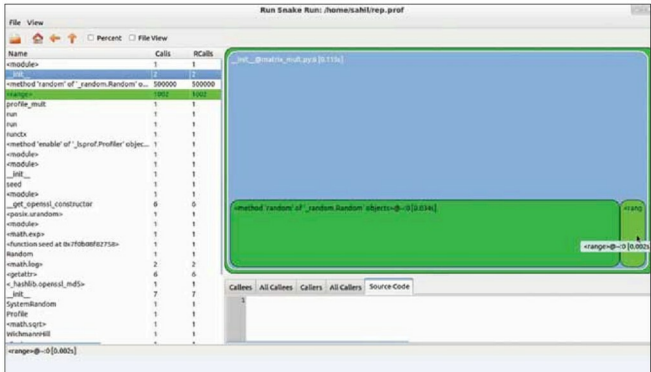


Figure 3: Runsnake